

INTRODUCTION AU SQL

ORDRE SELECT

Le SQL (Structured Query Language) est un langage informatique qui permet d'interagir avec des bases de données relationnelles. C'est le langage pour base de données le plus répandu, et c'est bien sûr celui utilisé par MySQL. Il a été créé dans les années 1970 et c'est devenu standard en 1986 (pour la norme ANSI - 1987 en ce qui concerne la norme ISO). Il est encore régulièrement amélioré.

I.1.2. Présentation succincte de MySQL

MySQL est donc un Système de Gestion de Bases de Données Relationnelles, qui utilise le langage SQL. C'est un des SGBDR les plus utilisés. Sa popularité est due en grande partie au fait qu'il s'agit d'un logiciel Open Source, ce qui signifie que son code source est librement disponible et que quiconque qui en ressent l'envie et/ou le besoin peut modifier MySQL pour l'améliorer ou l'adapter à ses besoins. Une version gratuite de MySQL est par conséquent disponible. À noter qu'une version commerciale payante existe également.

Création d'une BD

CREATE DATABASE NomDeLabase ;

Suppression d'une BD

DROP DATABASE NomDeLabase ;

DROP DATABASE IF EXISTS NomDeLabase ;

Création de table

```
CREATE TABLE [IF NOT EXISTS] NomDeLatable (  
  
    colonne1 description_colonne1,  
  
    colonne2 description_colonne2,  
  
    colonne3 description_colonne3,  
  
    ...  
  
    [PRIMARY KEY (colonne_clé_primaire)]  
  
)  
  
[ENGINE=moteur] ;
```

EXEMPLE

I.5.4.3. Vérifications

SHOW TABLES ; -- liste les tables de la base de données

DESCRIBE NomDeLaTable ; -- liste les colonnes de la table avec leurs caractéristiques

I.5.5. Suppression d'une table

La commande pour supprimer une table est la même que celle pour supprimer une base de données. Elle est, bien sûr, à utiliser avec prudence, car irréversible.

DROP TABLE NomDeLaTable ;

I.6. Modification d'une table

Lorsque l'on modifie une table, on peut vouloir lui ajouter, retirer ou modifier quelque chose.

Dans les trois cas, c'est la commande ALTER TABLE qui sera utilisée, une variante existant pour chacune des opérations :

ALTER TABLE NomDeLaTable

ADD ... -- permet d'ajouter quelque chose (une colonne par exemple)

ALTER TABLE NomDeLaTable

DROP ... -- permet de retirer quelque chose

ALTER TABLE NomDeLaTable

CHANGE ...

ALTER TABLE NomDeLaTable

MODIFY ... -- permettent de modifier une colonne

EXEMPLE

I.6.2. Ajout et suppression d'une colonne

I.6.2.1. Ajout On utilise la syntaxe suivante :

ALTER TABLE NomDeLaTable

ADD [COLUMN] nom_colonne description_colonne;

Le [COLUMN] est facultatif.

Description_colonne correspond à la même chose que lorsque l'on crée une table. Il contient le type de donnée et éventuellement NULL ou NOT NULL, etc.

I.6.2.2. Suppression La syntaxe de

ALTER TABLE ...

DROP ...

ALTER TABLE NomDeLaTable

DROP [COLUMN] nom_colonne; Comme pour les ajouts, le mot COLUMN est facultatif. Par défaut, MySQL considérera que vous parlez d'une colonne.

I.6.3. Modification de colonne

I.6.3.1. Changement du nom de la colonne

Vous pouvez utiliser la commande suivante pour changer le nom d'une colonne :

ALTER TABLE nom_table

CHANGE ancien_nom nouveau_nom description_colonne;

Par exemple, pour renommer la colonne nom en prenom, vous pouvez écrire

ALTER TABLE Test_tuto 2

CHANGE nom prenom VARCHAR(10) NOT NULL;

I.6.3.2. Changement du type de données

Les mots-clés CHANGE et MODIFY peuvent être utilisés pour changer le type de donnée de la colonne, mais aussi changer la valeur par défaut ou ajouter/supprimer une propriété AUTO_INCREMENT. Si vous utilisez CHANGE, vous pouvez, comme on vient de le voir, renommer la colonne en même temps. Si vous ne désirez pas la renommer, il suffit d'indiquer deux fois le même nom. Voici les syntaxes possibles :

```
ALTER TABLE NomDeLaTable
```

```
CHANGE ancien_nom nouveau_nom nouvelle_description;
```

```
ALTER TABLE NomDeLaTable
```

```
MODIFY nom_colonne nouvelle_description ;
```

I.7. Insertion de données

I.7.1. Syntaxe de INSERT

Deux possibilités s'offrent à nous lorsque l'on veut insérer une ligne dans une table : soit donner une valeur pour chaque colonne de la ligne, soit ne donner les valeurs que de certaines colonnes, auquel cas il faut bien sûr préciser de quelles colonnes il s'agit.

I.7.1.1. Insertion sans préciser les colonnes

EXEMPLE

I.7.1.3. Insertion multiple

Si vous avez plusieurs lignes à introduire, il est possible de le faire en une seule requête de la manière suivante :

INSERT INTO NomDeLaTable (Col1, Col2, Col3)

VALUES (Val11, Val12, Val13), (Val11, Val12, Val13), (Val11, Val12, Val13);

Bien entendu, vous êtes alors obligés de préciser les mêmes colonnes pour chaque entrée, quitte à mettre NULL pour certaines.

MODIFICATION DE DONNEES

Pour modifier les données d'une table (contenu), on utilise la syntaxe suivante

UPDATE NomTable

SET NomColonne=Valeur

WHERE Conditions;

Modifier l'inscription de l'étudiant M02 dans la classe L1RI.

I.8. Sélection de données

Comme son nom l'indique, ce chapitre traitera de la sélection et de l'affichage de données.

I.8.1. Syntaxe de l'Ordre SELECT

La requête qui permet de sélectionner et afficher des données s'appelle ORDRE SELECT.

SELECT permet donc d'afficher des données directement : des chaînes de caractères, des résultats de calculs, etc.

SYNTAXE

SELECT [DISTINCT] * | Col1 [AS Alias1], Col2 [AS Alias2], ...

FROM NomTable;

- **DISTINCT:** élimine les doublons
- *****: affiche toutes les colonnes
- **Col1, Col2,...:** indique les colonnes à afficher
- **Alias, Alias2,...:** attribue des alias aux colonnes

SELECT MatEtu "Matricule ", NomEtu "Nom Etudiant", PrenomEtu

- **FROM :**

Exemples

```
SELECT 'Bonjour à tous !' ;
```

```
SELECT 3+2 ;
```

SELECT permet également de sélectionner des données à partir d'une table. Pour cela, il faut ajouter une clause à la commande SELECT : la clause FROM, qui définit de quelle structure (dans notre cas, une table) viennent les données.

```
SELECT colonne1, colonne2, ...
```

```
FROM NomDeLaTable ;
```

EXEMPLE

Afficher le code et le nom de tous les clients :

```
Select NumCli, NomCli
```

```
From Client ;
```

Remarque :

EXEMPLE

Afficher le numéro, le nom et le stock de tous les produits.

```
Select NumPro, NomPro, StockPro
```


From Produit ;

- * affiche toutes les colonnes

Exemple

Afficher la liste de tous les clients.

Select *

From Client ;

Afficher le nom, le prénom et la ville de tous les clients. Choisissez des alias adéquats.

Select NomCli as « Nom », PrenomCli as « Prénom », VilleCli as « Ville »

From Client ;

Select *

From Client ;

EXERCICE

Afficher la liste de tous les clients.

Les requêtes SELECT faites jusqu'à présent sélectionnent toutes les lignes de la table. Or, bien souvent, on ne veut qu'une partie des données.

La clause WHERE

La clause WHERE ("où" en anglais) permet de restreindre (sélection) les résultats selon des critères (Prédicat) de recherche.

WHERE Condition(s) ;

EXEMPLE

Afficher le numéro, le nom et le stock de tous les produits dont le stock est inférieur à 100.

Select NumPro, NomPro, StockPro

Afficher pour chaque produit, le nom, le prix, le stock et la valeur du stock.

Insert into Contenir (CodePro, NumFac, PrixVente, Qte)

Values ('P01', 'F03', 1300, 8),

('P05', 'F04',1800,25);

From Produit

WHERE StockPro<100;

I.8.2.1. Les opérateurs de comparaison

Les opérateurs de comparaison sont les symboles que l'on utilise pour définir les critères de recherche. Huit opérateurs simples peuvent être utilisés. =, <, <=, >, >=, <> ou !=

EXEMPLE

Afficher le code, le nom et le stock de tous les produits dont le stock est inférieur à 100.

Select CodePro, NomPro, StockPro

From Produit

Where StockPro<100;

I.8.2.2. Combinaisons de critères

Pour cela, il faut des opérateurs logiques, qui sont au nombre de quatre : AND, && (AND), OR
|| (OR), XOR OU exclusif, NOT (NON) , ET AND

EXEMPLES

I.8.2.2.1. AND

C1 AND C2 vrai

EXEMPLE 1

**Afficher le numéro, le nom, le stock et le seuil de tous les produits qui ont atteint leur seuil
et dont le prix est inférieur à 1500.**

Select Numpro, nompro, stockpro, seuilpro

From produit

Where stockpro<=seuilpro AND PrixVente<1500;

I.8.2.2.2. OR

C1 OR C2

**Afficher le Numéro, le nom, le stock et le seuil de tous les produits qui ont atteint leur seuil
et les produits dont le seuil est égal à 0.**

Select codepro, nompro, stockpro, seuilpro

From produit

Where stockpro<=seuilpro OR stockPro=0;

EXEMPLE

Afficher le code, le nom, le stock et le seuil de tous les produits qui ont atteint leur seuil et les produits dont le stock est compris entre 100 et 200.

Select NumPro, nompro, stockpro, seuilpro

From produit

Where stockpro<=seuilpro OR stockpro>=100 AND stockpro<=200);

EXEMPLE

Famille (NumFam, NomFam)

Produit (NimPro, NomPro, PrixVente, StockPro, SeuilPro, #NumFam)

EXERCICE

Créer la table Famille

Create Table Famille(

NumFam Varchar(4) Primiry Key,

NomFam varchar(20)) ;

Modifions la table Produit (ajout de la propriété NumFam)

Alter table Produit

Add NumFam varchar(4) references Famille(NumFam);

Afficher le nom et le stock de tous les produits des familles F01 et F03 qui ont un stock inférieur à 100.

Select nompro, stockpro

From produit

Where stockpro<=100 AND (CodeFam='FA01' OR CodeFam='F03');

Exemples

I.8.2.2.3. NOT

Exemples

I.8.2.2.4. XOR

Exemples

I.8.2.3. Sélection complexe

Lorsque vous utilisez plusieurs critères, et que vous devez donc combiner plusieurs opérateurs logiques, il est extrêmement important de bien structurer la requête. En particulier, il faut placer des parenthèses au bon endroit. En effet, cela n'a pas de sens de mettre plusieurs opérateurs logiques différents sur un même niveau.

Exemples

I.8.2.4. Le cas de NULL (IS NULL)

EXEMPLE

Afficher la liste de tous les clients qui n'ont pas de ville.

Select *

From client

Where villecli IS NULL;

I.8.3. Tri des données

Pour trier vos données, c'est très simple, il suffit d'ajouter **ORDER BY** qui trie les ramenées par la requête (après les critères de sélection de WHERE s'il y en a).

Exemples

I.8.3.1. Tri ascendant ou descendant

Pour déterminer le sens du tri effectué, SQL possède deux mots-clés : **ASC** pour ascendant, et **DESC** pour descendant. Par défaut, si vous ne précisez rien, c'est un tri ascendant qui est effectué :

Exemple

I.8.3.2. Trier sur plusieurs colonnes

Il est également possible de trier sur plusieurs colonnes. L'ordre dans lequel vous donnez les colonnes est important, le tri se fera d'abord sur la première colonne donnée, puis sur la seconde, etc. Vous pouvez trier sur autant de colonnes que vous voulez.

SYNTAXE

ORDER BY COL1 ASC|DESC, COL2 ASC|DESC

EXEMPLE

Afficher la liste des produits triée par ordre croissant des prix et décroissant sur le nom des produits.

Select *

From produit

Order by PrixUnitaire ASC, Nompro DESC;

EXEMPLE 2

Afficher les dix produits les plus chers.

Select *

From produit

Order by PrixUnitaire DESC

LIMIT 10 ;

I.8.4. Éliminer les doublons

Il peut arriver que MySQL vous donne plusieurs fois le même résultat tout simplement parce que certaines informations sont présentes plusieurs fois dans la table. Pour éliminer les doublons, on utilise alors le mot-clé DISTINCT.

Exemple

I.8.5. Restreindre les résultats

En plus de restreindre une recherche en lui donnant des critères grâce à la clause WHERE, il est possible de restreindre le nombre de lignes récupérées. Cela se fait grâce à la clause LIMIT.

I.8.5.1. Syntaxe LIMIT s'utilise avec deux paramètres.

- Le nombre de lignes que l'on veut récupérer.
- Le décalage, introduit par le mot-clé OFFSET et qui indique à partir de quelle ligne on récupère les résultats. Ce paramètre est facultatif. S'il n'est pas précisé, il est mis à 0.

```
SELECT *  
  
FROM Animal  
  
ORDER BY id  
  
LIMIT 6,
```

I.9. Élargir les possibilités de la clause WHERE

Dans le chapitre précédent, vous avez découvert la commande SELECT, ainsi que plusieurs clauses permettant de restreindre et d'ordonner les résultats selon différents critères. Jusqu'ici, les conditions permises par WHERE étaient très basiques. Mais cette clause offre bien d'autres possibilités parmi lesquelles :

- **LIKE** : la comparaison avec une valeur incomplète (chercher les animaux dont le nom commence par une certaine lettre par exemple) ;
- **BETWEEN** : la comparaison avec un intervalle de valeurs (entre 2 et 5 par exemple) ;

SYNTAXE

Val BETWEEN Borne_Min AND Borne_Max;

EXEMPLE:

Afficher la liste des produits dont le prix est compris entre 1500 et 3000.

SELECT *

FROM PRODUIT

WHERE PrixUnitaire BETWEEN 1500 AND 3000 ;

EXEMPLE

Afficher le numéro et la date de toutes les factures émises entre le 10-10-2022 et le 31-12-2022.

Le résultat sera trié par croissant sur les date et décroissant sur les numéros.

— la comparaison avec un ensemble de valeurs (comparaison avec 5, 6, 10 ou 12 par exemple).

I.9.1. Recherche approximative

Cette recherche est faite en particulier pour des recherches sur des chaînes de caractères. En effet, comment faire lorsqu'on ne sait pas si le mot que l'on recherche est au singulier ou au pluriel par exemple ? Ou si l'on cherche toutes les lignes dont le champ "commentaires" contient un mot particulier ? Pour ce genre de recherches, l'opérateur **LIKE** est très utile, car

il permet de faire des recherches en utilisant des "jokers", c'est-à-dire des caractères qui représentent n'importe quel caractère.

Deux jokers existent pour LIKE :

— '%' : qui représente n'importe quelle chaîne de caractères, quelle que soit sa longueur (y compris une chaîne de longueur 0) ;

— '_' : qui représente un seul caractère (ou aucun). Quelques exemples :

— 'b%' cherchera toutes les chaînes de caractères commençant par 'b' ('brocoli', 'bouli', 'b')

— 'B_' cherchera toutes les chaînes de caractères contenant une ou deux lettres dont la première est 'b' ('ba', 'bf', 'b')

— '%ch%ne' cherchera toutes les chaînes de caractères contenant 'ch' et finissant par 'ne'

('chne', 'chine', 'échine', 'le pays le plus peuplé du monde est la Chine')

%L_

— '_ch_ne' cherchera toutes les chaînes de caractères commençant par 'ch', éventuellement précédées d'une seule lettre, suivies de zéro ou d'un caractère au choix et enfin se terminant par 'ne' ('chine', 'chne', 'echine')

Rechercher '%' ou '_' Comment faire si vous cherchez une chaîne de caractères contenant '%' ou '_' ? Évidemment, si vous écrivez LIKE '%' ou 'LIKE_', MySQL vous donnera absolument toutes les chaînes de caractères dans le premier cas, et toutes les chaînes de 0 ou 1 caractère dans le deuxième. Il faut donc signaler à MySQL que vous ne désirez pas utiliser « % » ou « _ » tant que joker, mais bien en tant que caractère de recherche. Pour ça, il suffit de mettre le caractère d'échappement '\\', dont je vous ai déjà parlé, devant le '%' ou le '_'.

Exemple :

```
SELECT *
```

```
FROM Client
```

```
WHERE Nomcli LIKE '%%%' ESCAPE '\';
```

I.9.1.0.2. Exclure une chaîne de caractères

C'est logique, mais je précise quand même (et puis ça fait un petit rappel) : l'opérateur logique

NOT est utilisable avec LIKE

I.9.1.1. Sensibilité à la casse

Vous l'aurez peut-être remarqué en faisant des essais, LIKE 'chaîne de caractères' n'est pas sensible à la casse (donc aux différences majuscules-minuscules). Pour rappel, ceci est dû à l'interclassement. Nous avons gardé l'interclassement par défaut du jeu de caractère UTF-8, qui n'est pas sensible à la casse. Si vous désirez faire une recherche sensible à la casse, vous pouvez définir votre chaîne de recherche comme une chaîne de type binaire, et non plus comme une simple chaîne de caractères :

Afficher la liste de tous les clients dont le nom contient la lettre M

I.9.1.2. Recherche dans les numériques

Vous pouvez bien entendu utiliser des chiffres dans une chaîne de caractères. Après tout, ce sont des caractères comme les autres. Par contre, utiliser LIKE sur un type numérique (INT par exemple), c'est déjà plus étonnant. Et pourtant, MySQL le permet. Attention cependant, il

s'agit bien d'une particularité MySQL, qui prend souvent un malin plaisir à étendre la norme SQL pure. LIKE '1%' sur une colonne de type numérique trouvera donc des nombres comme 10, 1000, 153 1 SELECT * 2 FROM Animal 3 WHERE id LIKE '1%';

I.9.2. Recherche dans un intervalle

Il est possible de faire une recherche sur un intervalle à l'aide uniquement des opérateurs de comparaison >= et <=. Cependant, SQL dispose d'un opérateur spécifique pour les intervalles, qui pourrait vous éviter les erreurs d'inattention classiques (< au lieu de > par exemple) en plus de rendre votre requête plus lisible et plus performante : BETWEEN minimum AND maximum (between signifie "entre" en anglais).

FONCTIONS DE GROUPES

- **COUNT(Colonne) :**
- **Count(*) :**
- **MIN(colonne) :**
- **- MAX(Colonne) :**
- **AVG(Colonne)**
- **-SUM(Colonne) :**

LES JOINTURES

Exemple

ETUDIANT (Matricule, Nom, Prenom, DateNaiss)

CLASSE (Numero, NomClasse, Mensualite, DroitIns)

INSCRIPTION (CodeCla, Matricule, DateIns, AnneeAca)

- 1) **Afficher les inscriptions de l'année académique 2022-2023.**
- 2) **Afficher la liste des classes dont la mensualité est supérieure à 50000 avec des droits d'inscription compris entre 100000 et 150000.**
- 3) **Afficher la liste des étudiants dont les noms commencent par MA.**
- 4) **Afficher tous les étudiants dont le nom commence par M et se termine par E.**
- 5) **Afficher la liste des cinq classes ayant les plus grandes mensualités.**